

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA17

ANALYTICAL PROBLEM SOLVING

1. Water Jug Problem

You are given 2 jugs, a 4-gallon one and a 3-gallon one. Neither has any measuring marker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into the 4-gallon jug?

Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug into another, and no other measuring devices are available.

Problem

You have a 4-gallon jug and a 3-gallon jug, both unmarked. Using only filling, emptying, and pouring, obtain exactly 2 gallons of water in the 4-gallon jug.

Solution Approach

Instead of starting with the 3-gallon jug, this solution starts by filling the 4-gallon jug first and transfers water into the 3-gallon jug to isolate the 2-gallon quantity.

Steps

- Fill the 4-gallon jug completely. $\rightarrow (4G = 4, 3G = 0)$
- Pour from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full. $\rightarrow (4G = 1, 3G = 3)$
- Empty the 3-gallon jug onto the ground. $\rightarrow (4G = 1, 3G = 0)$
- Pour the remaining 1 gallon from the 4-gallon jug into the 3-gallon jug. $\rightarrow (4G = 0, 3G = 1)$
- Fill the 4-gallon jug again. $\rightarrow (4G = 4, 3G = 1)$
- Pour from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full (it needs only 2 more gallons). $\rightarrow (4G = 2, 3G = 3)$

In Tabular Form

Step	Action	4-Gallon Jug	3-Gallon Jug
1	Fill the 4-gallon jug completely	4	0
2	Pour from the 4-gallon jug into the 3-gallon jug until full	1	3
3	Empty the 3-gallon jug onto the ground	1	0
4	Pour the remaining 1 gallon into the 3-gallon jug	0	1
5	Fill the 4-gallon jug again	4	1
6	Pour from the 4-gallon jug into the 3-gallon jug until full	2	3

Final State

- 4-gallon jug = 2 gallons
- 3-gallon jug = 3 gallons

Therefore, exactly 2 gallons of water are obtained in the 4-gallon jug, reached through a different pouring sequence than starting with the 3-gallon jug.

2. Mars Rover – Intelligent Agent Analysis

Find out how the Mars Rover analyses and explores samples on Mars by answering: (i) percepts, (ii) operating environment, (iii) actions, (iv) performance evaluation, (v) most suitable agent architecture.

Problem Statement

A Mars Rover is an autonomous robotic explorer deployed on the Martian surface to study terrain, collect samples, and transmit findings to Earth. Because signal delay between Earth and Mars makes real-time human control impossible, the rover must function as an intelligent agent capable of sensing, reasoning, and acting independently.

i. Percepts of the Agent

Percepts are the raw inputs the rover gathers from its sensors before any decision is made. Typical percepts include:

- Visual feed from onboard cameras
- Mineral and chemical composition readings
- Ground and ambient temperature values
- Barometric pressure of the Martian atmosphere
- Dust and wind-speed sensor data
- Proximity/obstacle-detection sensor output
- Slope and terrain-texture information
- Remaining battery charge and solar input

In the same way a human relies on sight and touch to understand surroundings, the rover relies on its instrument suite to build a picture of the Martian environment before choosing an action.

ii. Characterization of the Operating Environment

- Partially Observable – onboard sensors only reveal the rover's immediate vicinity, not the whole planet.
- Dynamic – dust storms, temperature swings, and shifting terrain can change conditions between observations.
- Sequential – each movement or sampling decision constrains what the rover can do next.
- Continuous – position, speed, and sensor readings vary over a continuous range, not discrete steps.
- Single-Agent – the rover operates without competing or cooperating agents in its immediate task.

Because Mars is largely uncharted, the rover must continually revise its understanding of the environment as new data arrives.

iii. Actions Available to the Agent

- Drive forward or reverse
- Steer left or right
- Halt movement
- Capture images of the surroundings
- Extract rock samples
- Extract soil samples
- Run onboard analysis on collected samples
- Drill beneath the surface
- Steer around detected obstacles
- Transmit collected data to Earth
- Reorient solar panels to recharge

Each action is chosen based on current sensor readings so that the rover both progresses its mission and avoids damage.

iv. Performance Evaluation

- Scientific Yield – number and quality of useful samples and readings collected
- Ground Coverage – total distance traveled and new terrain explored
- Communication Reliability – completeness and accuracy of data relayed to Earth
- Power Management – efficiency of battery and solar-energy usage
- Operational Safety – avoidance of collisions, falls, or terrain hazards

A rover is judged successful when it maximizes scientific return while conserving power and avoiding damage over an extended mission life.

v. Most Suitable Agent Architecture

Model-Based Utility-Based Agent

A purely reflex or goal-based design is not sufficient here, because the rover must weigh several competing options (which sample to collect, which path to take) rather than simply detect a goal condition. A model-based utility agent keeps an internal model of the terrain already explored and assigns a utility score to each candidate action based on scientific value, safety, and energy cost, then selects the action with the highest expected utility.

Advantages

- Combines memory of past terrain with forward-looking evaluation of choices
- Copes well with uncertainty caused by partial observability
- Balances competing priorities such as safety versus scientific reward
- Adapts its plan as new percepts revise its internal model

Conclusion

The Mars Rover senses its surroundings through a range of instruments, reasons over an internal model of the terrain, and acts to balance exploration, safety, and power usage. Given the partially observable, dynamic, and continuous nature of Mars, a Model-Based Utility-Based Agent architecture offers the best fit for autonomous operation.

3. The 8-Queens Problem – Search Space Formulation

Place 8 queens on a chessboard such that none of the queens attack any of the others, and formulate the problem components using state-space search.

Problem Statement

The 8-Queens Problem asks for a placement of 8 queens on an 8×8 chessboard so that no two queens share a row, column, or diagonal. Two queens attack each other if they lie on the same row, the same column, or the same diagonal, so a valid solution must avoid all three conditions simultaneously.

Problem Formulation Components

1. Initial State

The search begins with an empty 8×8 board — no queens have been placed yet.

2. State Space

A state is any partial or complete placement of queens on the board, described by which columns already hold a queen and in which row. The full state space is the set of every possible arrangement of up to 8 queens across the 64 squares.

3. Actions (Operators)

- Add a queen to the next unfilled column in a row that is not attacked by any queen already placed

- Backtrack and try a different row if no safe row exists in the current column

For example: place Queen 1 in column 1, then place Queen 2 in column 2 only in a row that does not conflict with Queen 1, and continue column by column.

4. Goal Test

A state satisfies the goal test when all 8 queens are placed and:

- No two queens share a row
- No two queens share a column
- No two queens share a diagonal

5. Path Cost

Each placement step is assigned a uniform cost of 1. Since every solution requires exactly 8 placements, path cost plays little role — the objective is simply to reach any valid goal state.

Search Strategy Used: Backtracking Search

- Place a queen in the current column's safest available row
- Move to the next column and repeat
- If a column has no safe row, backtrack to the previous column and try another row
- Continue until all 8 queens are placed without conflict

Example of a Valid Solution

One valid, non-attacking arrangement (column → row) is:

Column	1	2	3	4	5	6	7	8
n								
Row	4	6	8	2	7	1	3	5

```

-- -- Q -- -- -- (row 4, column 1)
-- -- -- -- Q -- (row 6, column 2)
-- -- -- -- -- Q (row 8, column 3)
-- Q -- -- -- -- (row 2, column 4)
-- -- -- -- -- Q -- (row 7, column 5)
Q -- -- -- -- -- (row 1, column 6)
-- Q -- -- -- -- (row 3, column 7)
-- -- -- Q -- -- (row 5, column 8)

```

No two queens in this layout share a row, column, or diagonal.

Performance Measure

- Whether a valid, non-attacking configuration is reached
- Number of states expanded before success
- Time taken to reach the solution
- Memory consumed while searching

Conclusion

The 8-Queens problem is a classic state-space search task where the initial state is an empty board, actions place queens column by column, and the goal test requires all 8 queens to coexist without conflict. Backtracking Search efficiently solves it by abandoning unsafe partial placements early, and the arrangement (4, 6, 8, 2, 7, 1, 3, 5) is one of the valid solutions it can find.

4. OLA Cab Booking – Problem-Solving Agent

A customer wants to travel from one location to another using the OLA Cab booking mobile application, choosing among mini, micro, sedan, shared, prime, and similar options. Identify the type of problem-solving agent involved and write pseudocode for the process.

Problem Statement

A customer wants to travel from a starting point to a destination using the OLA Cab app, selecting from cab categories such as Micro, Mini, Sedan, Prime, and Shared. The system must guide the customer toward the option that best balances fare, comfort, and waiting time.

Type of Agent Used

Utility-Based Problem Solving Agent

Reason

- Simply reaching the destination is not the only concern — the customer weighs fare, comfort, and wait time against each other.
- The agent assigns a utility score to each available cab option rather than accepting the first one that satisfies the goal.
- It recommends the cab type with the highest combined utility for the customer's stated preferences.
- This makes it a Utility-Based Agent rather than a purely Goal-Based one, since multiple valid options exist and must be ranked, not just found.

Thus, the OLA booking system behaves as a Utility-Based Problem Solving Agent.

Problem Formulation Components

Initial State

- Customer opens the OLA application
- Current GPS location is detected

Actions

- Enter the destination
- Retrieve the list of nearby available cabs
- Score each cab option on fare, comfort, and wait time
- Recommend / select the highest-utility cab
- Confirm the booking

Goal State

- Customer is picked up and delivered to the destination in the cab offering the best overall utility

Path Cost / Utility Factors

- Fare amount
- Estimated travel time
- Estimated waiting time
- Comfort rating of the cab category

Pseudocode

Algorithm OLA_CAB_BOOKING_UTILILITY

Start

1. Open OLA Application

```

2. Detect Current Location
3. Enter Destination
4. Fetch Available Cabs: {Micro, Mini, Sedan, Prime, Shared}
5. For each Cab in Available Cabs
    Compute Utility(Cab) = w1*Comfort - w2*Fare - w3*WaitTime
6. Select Cab with Maximum Utility
7. If Selected Cab is Available Then
    Calculate Final Fare
    Confirm Booking
    Assign Driver
    Start Trip
    Reach Destination
    Display "Trip Completed Successfully"
Else
    Recompute Utility Excluding Unavailable Cab
    Repeat Step 6
End If
Stop

```

Working Explanation

- The customer opens the app and the system detects the pickup point
- The destination is entered and nearby cab options are fetched
- Each cab type is scored using a utility function of comfort, fare, and wait time
- The cab with the highest utility is proposed to the customer
- If unavailable, the next-best utility option is chosen automatically
- Once confirmed, the trip proceeds until the destination is reached

Conclusion

The OLA Cab Booking system is best modeled as a Utility-Based Problem Solving Agent because it must rank several competing cab options by a combination of fare, comfort, and wait time rather than simply satisfying a single goal condition.

5. Logistics Network – Uniform Cost Search (UCS)

A logistics company must move packages between warehouses at minimum cost across a weighted graph. Determine the least-cost path from the starting warehouse S to the destination warehouse G using Uniform Cost Search.

Weighted Delivery Network Used

- S → A, cost 1
- S → B, cost 6
- A → C, cost 4
- A → D, cost 2
- B → D, cost 1
- C → G, cost 1
- D → G, cost 2

Uniform Cost Search (UCS) – Step-by-Step Table

Step	Node Expanded	Cost	New Nodes Added
1	S	0	A(1), B(6)
2	A	1	C(5), D(3)
3	D	3	G(5)
4	C	5	G(6) – not an improvement
5	G	5	Goal Reached

Possible Paths and Costs

Path	Cost
S → B → D → G	6 + 1 + 2 = 9
S → A → C → G	1 + 4 + 1 = 6
S → A → D → G	1 + 2 + 2 = 5

Result

Item	Value
Start Node	S
Goal Node	G
Least Cost Path	S → A → D → G
Total Cost	5

Answer

Optimal Path: S → A → D → G

Minimum Cost: 1 + 2 + 2 = 5